A PRACTITIONER'S GUIDE

Agentic AI with LangGraph

From Graph Fundamentals to Production-Ready Multi-Agent Systems

Part 1: Foundations — Chapters 1–3

CHAPTER 1

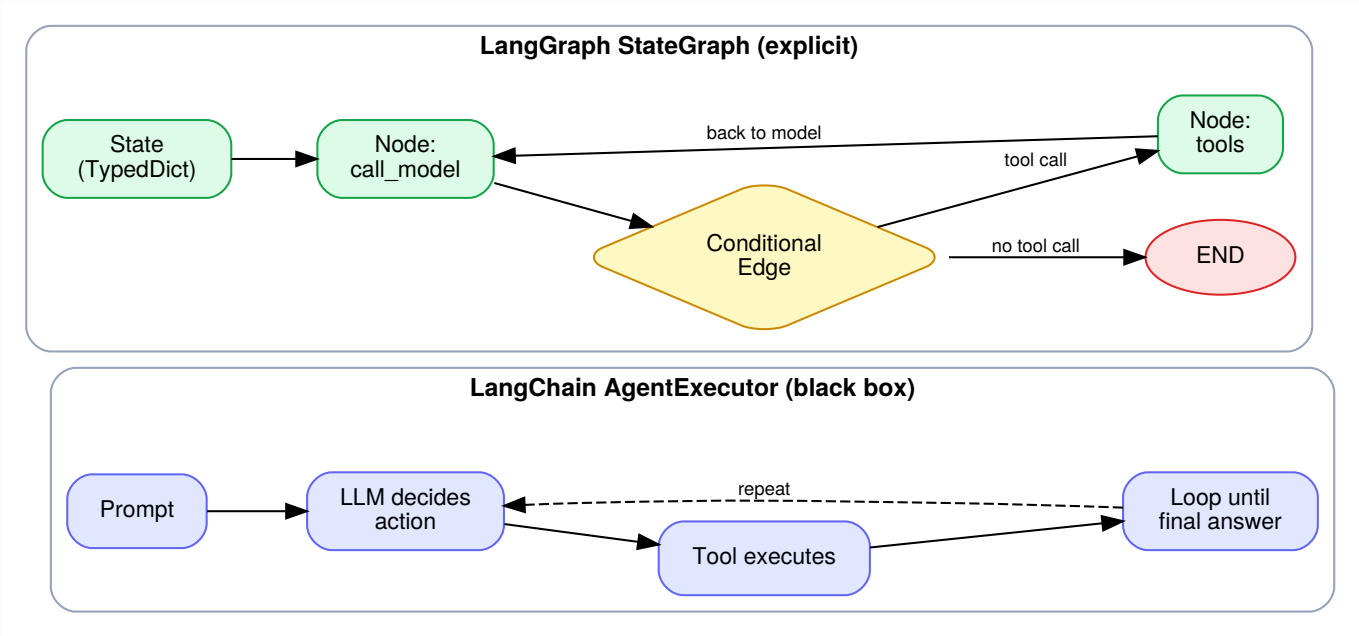
From Chains to Graphs

1.1 Conceptual Overview

If you've worked with LangChain, you already know its core idea: compose LLM calls, prompts, tools, and parsers into a **chain**, and for agents, hand control to an `AgentExecutor` that loops internally — decide, act, observe, repeat — until it thinks it's done.

That loop works, but it is a **black box**. You cannot easily pause it mid-thought, inspect exactly what happened at step 3, force a human to approve step 4 before it fires, or resume it after your server crashed. The control flow lives inside library code, not your code.

LangGraph inverts this. Instead of an executor that hides the loop, you explicitly draw the loop as a **graph**: a set of **nodes** (units of work — usually a Python function) connected by **edges** (transitions, which can be conditional). A shared **state** object flows through the graph, and every node reads from it and writes back to it.



> **Mental model:** LangChain's AgentExecutor is a while-loop you don't control. A LangGraph `StateGraph` is a state machine you draw yourself — every transition, branch, and loop-back is a line of code you wrote and can inspect, checkpoint, or interrupt.

1.1.1 Why this matters for "agentic" AI specifically

An "agent" is really just a system where *the control flow itself is decided by the LLM* — not a fixed pipeline. That means your program needs cycles (the model might call a tool, look at the result, and decide to call another tool), and it needs durable state (the agent might run for minutes or hours, get interrupted, and need to resume). LangChain's chain abstraction (a DAG, no cycles) was never built for this. LangGraph is a low-level orchestration layer purpose-built for cyclic, stateful, multi-step LLM applications — which is exactly what "agentic AI" means in practice.

1.1.2 LangGraph is built on top of LangChain, not a replacement for it

You still use LangChain's chat model wrappers (`ChatOpenAI` , `ChatAnthropic` , etc.), its `Tool` definitions, its prompt templates, and its message types (`HumanMessage` , `AIMessage` , `ToolMessage`). LangGraph only replaces the *orchestration* layer — the part that decided what runs next.

Concern	LangChain	LangGraph
Unit of composition	Runnable / Chain	Node (function) + Edge
Control flow	Linear or hidden inside AgentExecutor	Explicit graph, supports cycles
State	Implicit / passed through chain inputs	Explicit typed State object

Persistence	Manual	Built-in checkpointers
Human-in-the-loop	Custom, ad hoc	Native `interrupt` support
Best for	Simple pipelines, RAG chains	Agents, multi-step workflows, multi-agent systems

1.1.3 When to actually reach for LangGraph

- The number of steps isn't fixed in advance — the model decides how many tool calls to make.
- You need the run to survive a process restart (checkpointing / durability).
- You need a human to approve or edit something mid-run.
- You're coordinating more than one "agent" (e.g. a supervisor delegating to specialists).
- You need to debug or visualize exactly what the agent did, step by step.

If your use case is "prompt in, single LLM call, answer out" or a strictly linear RAG pipeline, plain LangChain (or even no framework) is simpler and you don't need LangGraph's machinery.

1.2 Implementation: Setting Up

Install the library in a virtual environment (Python 3.9+):

```
python -m venv venv
source venv/bin/activate        # Windows: venv\Scripts\activate

pip install -U langgraph langchain langchain-openai
```

Sanity-check the install:

```
import langgraph
print(langgraph.__version__)
```

Set your model provider API key as an environment variable (e.g. `OPENAI_API_KEY` or `ANTHROPIC_API_KEY`) before running any of the examples in this guide.

> **Note:** LangGraph also ships a JS/TS package (`@langchain/langgraph`) with an equivalent API. This guide uses Python throughout.

1.3 Interview Questions

Q1. What problem does LangGraph solve that LangChain's AgentExecutor doesn't?

A: AgentExecutor hides the reasoning loop inside library code — you can't easily inspect intermediate state, persist a run, pause for human approval, or resume after a crash. LangGraph makes the loop an explicit graph of nodes and edges over a shared state object, giving you full control over control flow, persistence (via checkpointers), and interruption points.

Q2. Is LangGraph a replacement for LangChain?

A: No — it's built on top of it. You still use LangChain's model wrappers, tools, prompts, and message types. LangGraph only replaces the orchestration/control-flow layer, which is why the two are typically used together, not as alternatives.

Q3. Why do agentic systems fundamentally need cycles, and why couldn't a DAG-based chain support that?

A: An agent's number of steps is data-dependent — the LLM decides whether to call another tool after seeing a result, which requires looping back to the same decision point (e.g., "call model" → "run tool" → "call model" again). A DAG, by definition, has no cycles, so a chain-based pipeline can't natively express "keep going until the model says stop." A graph with conditional edges can.

Q4. Give two concrete scenarios where you'd choose LangGraph over a simple LangChain chain.

A: (1) A customer-support agent that must pause and wait for a human to approve a refund above $100 before continuing (human-in-the-loop). (2) A research agent that might run for several minutes across many tool calls and needs to survive a server restart without losing progress (checkpointed persistence).

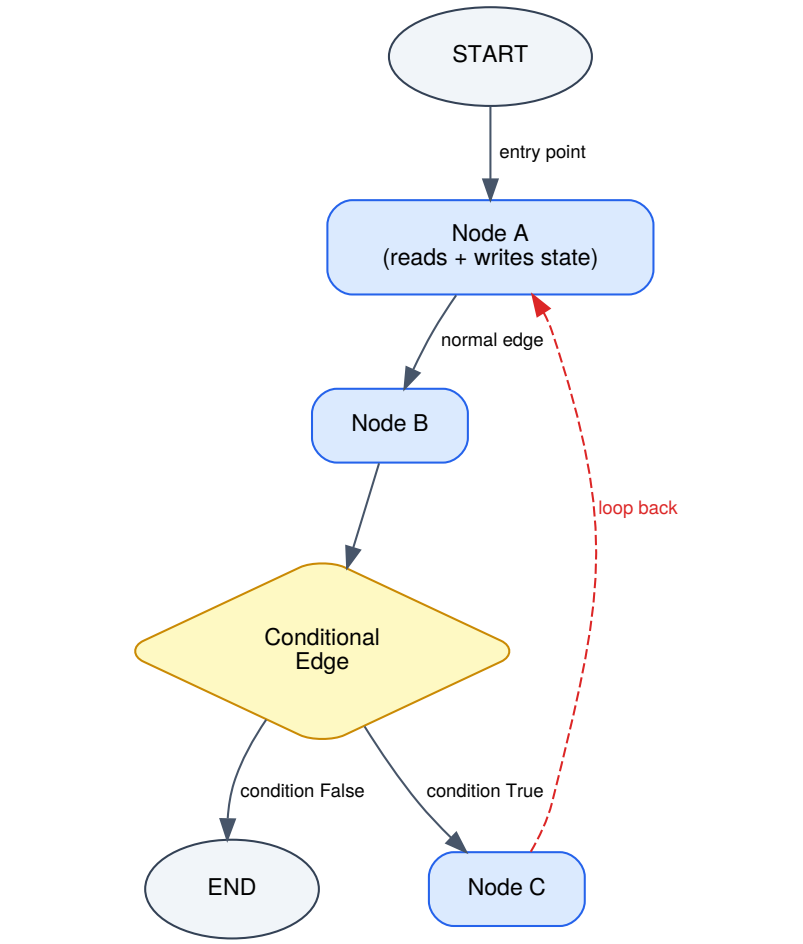
CHAPTER 2

Core Concepts: State, Nodes, Edges

2.1 Conceptual Overview

Every LangGraph application is built from four ingredients:

1. **State** — a schema (usually a `TypedDict`) describing the data that flows through the graph.
2. **Nodes** — plain Python functions that take the current state and return a partial update to it.
3. **Edges** — connections that say "after this node, go to that node." Edges can be fixed or **conditional** (a function decides the next node at runtime).
4. **The compiled graph** — once nodes and edges are registered on a `StateGraph` builder, calling `.compile()` produces a runnable object.



Anatomy of a StateGraph: State flows through nodes; edges (normal or conditional) decide the next node.

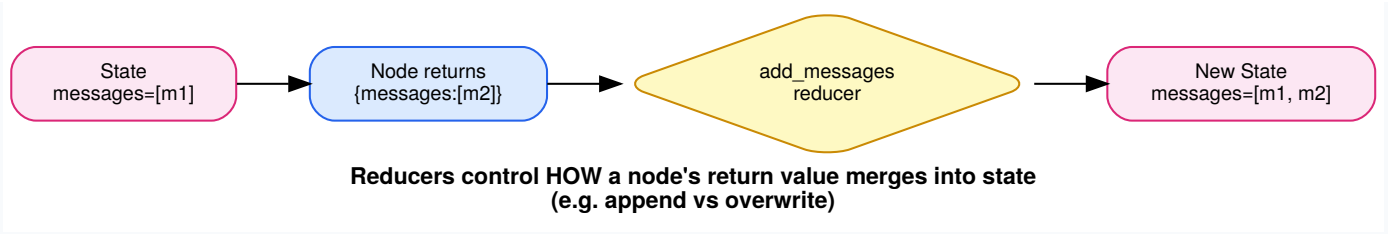
2.1.1 Defining State

State is typically a `TypedDict`. Each field can optionally declare a **reducer** function via `Annotated`, which controls how a node's returned value is merged into the existing state, rather than simply overwriting it.

```
from typing import TypedDict, Annotated
from langgraph.graph.message import add_messages

class AgentState(TypedDict):
    messages: Annotated[list, add_messages]   # appends, doesn't overwrite
    step_count: int                           # default behaviour: overwrite
```

Without a reducer, LangGraph's default behaviour is to **overwrite** the field with whatever the node returns. This is wrong for something like a message history, where you want to *append* the new message rather than replace the whole list — hence the built-in `add_messages` reducer.



2.1.2 Nodes

A node is any callable that accepts the state (and optionally a config object) and returns a *dict of updates* — not the full state, just the keys it wants to change.

```
def call_model(state: AgentState) -> dict:
    response = llm.invoke(state["messages"])
    return {"messages": [response]}   # merged via add_messages reducer
```

2.1.3 Edges

There are three kinds of edges:

Edge type	Method	Behaviour
Normal	`graph.add_edge("A", "B")`	Always go from A to B
Conditional	`graph.add_conditional_edges("A", router_fn)`	`router_fn(state)` returns the name of the next node at runtime
Entry / Finish	`graph.set_entry_point("A")` / `END`	Marks where execution starts / can terminate

Note: Newer LangGraph versions also support returning a `Command` object directly from a node to combine a state update and a routing decision in one place, instead of a separate conditional-edge function. Both styles are valid; conditional edges are more explicit and easier to visualize, which is why we use them throughout this guide.

2.2 Implementation

```
from langgraph.graph import StateGraph, START, END

builder = StateGraph(AgentState)

builder.add_node("call_model", call_model)
builder.add_node("call_tool", call_tool)

builder.add_edge(START, "call_model")
builder.add_conditional_edges(
    "call_model",
    lambda state: "call_tool" if needs_tool(state) else END,
)
builder.add_edge("call_tool", "call_model")

graph = builder.compile()
```

Once compiled, you run it like any other object with an `.invoke()`, `.stream()`, or `.ainvoke()` method:

```
result = graph.invoke({"messages": [("user", "What's 2+2?")]})
print(result["messages"][-1].content)
```

2.3 Interview Questions

Q1. What is a reducer in LangGraph, and why is the default merge behaviour insufficient for message history?

A: A reducer is a function attached to a state field (via `Annotated[type, reducer_fn]`) that controls how a node's returned value combines with the existing value for that field. The default behaviour is overwrite — a node returning `{"messages": [new_msg]}` would replace the entire message list rather than appending to it. Message history needs an append-style reducer (`add_messages`), which also handles deduplication by message ID and updates to existing messages.

Q2. What's the difference between a normal edge and a conditional edge?

A: A normal edge (`add_edge`) always transitions from node A to node B. A conditional edge (`add_conditional_edges`) is attached to a routing function that inspects the current state at runtime and returns the name of whichever node should run next — this is what makes branching and looping possible.

Q3. Does a node return the full updated state or just a partial update? Why does that design choice matter?

A: A node returns only the keys it wants to change, as a dict. LangGraph merges this partial update into the full state using each field's reducer. This matters because it keeps nodes simple and decoupled — a node doesn't need to know about, or accidentally clobber, state fields it isn't responsible for.

Q4. What happens if you don't call `.compile()` on a StateGraph builder?

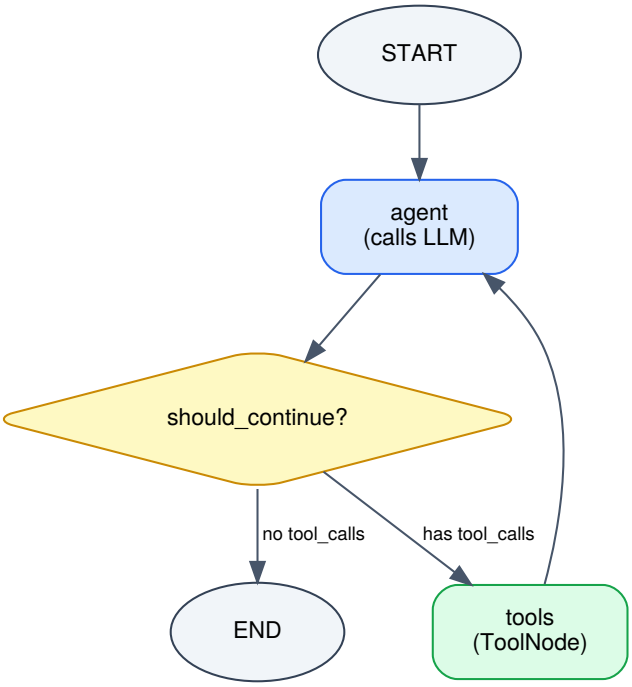
A: The graph builder (`StateGraph` instance) is just a specification — it has no `.invoke()` / `.stream()` methods and cannot run. `.compile()` validates the graph (e.g., checks every node is reachable and edges reference real node names) and produces an executable, LangChain-Runnable-compatible object.

CHAPTER 3

Your First Agent Graph

3.1 Conceptual Overview

We'll now assemble Chapter 2's pieces into a complete, runnable ReAct-style agent: a loop of "call the model → did it ask for a tool? → run the tool → call the model again → ... → done."



Minimal ReAct-style agent graph built in Chapter 3

The graph has exactly two real work nodes — `agent` and `tools` — and one conditional router (`should_continue`) that inspects the model's latest response: if it contains tool calls, route to `tools`; otherwise route to `END`. After `tools` runs, control always flows back to `agent` so the model can see the tool's result and decide what to do next.

3.2 Implementation

```
from typing import TypedDict, Annotated
from langgraph.graph import StateGraph, START, END
from langgraph.graph.message import add_messages
from langgraph.prebuilt import ToolNode
from langchain_openai import ChatOpenAI
from langchain_core.tools import tool

# 1. Define a tool
@tool
def get_weather(city: str) -> str:
    """Look up the current weather for a city."""
    return f"It's sunny in {city}."

tools = [get_weather]
llm = ChatOpenAI(model="gpt-4o").bind_tools(tools)

# 2. Define state
class AgentState(TypedDict):
    messages: Annotated[list, add_messages]

# 3. Define nodes
def agent_node(state: AgentState) -> dict:
    response = llm.invoke(state["messages"])
    return {"messages": [response]}

tool_node = ToolNode(tools)   # prebuilt: runs any requested tool calls

# 4. Define the router
```

```
def should_continue(state: AgentState) -> str:
    last_message = state["messages"][-1]
    if last_message.tool_calls:
        return "tools"
    return END

# 5. Wire the graph
builder = StateGraph(AgentState)
builder.add_node("agent", agent_node)
builder.add_node("tools", tool_node)
builder.add_edge(START, "agent")
builder.add_conditional_edges("agent", should_continue)
builder.add_edge("tools", "agent")

graph = builder.compile()

# 6. Run it
result = graph.invoke({
    "messages": [("user", "What's the weather in Tokyo?")]
})
print(result["messages"][-1].content)
```

Note: `ToolNode` is a prebuilt LangGraph node — it automatically reads `tool_calls` off the last AI message, executes the matching tool(s), and returns the results wrapped as `ToolMessage` objects. You don't have to hand-write the dispatch logic yourself.

3.2.1 Visualizing the compiled graph

```
# In a Jupyter notebook:
from IPython.display import Image
Image(graph.get_graph().draw_mermaid_png())
```

This is extremely useful for sanity-checking that the graph you built in code matches the graph you had in your head — always worth doing before debugging runtime behaviour.

3.3 Interview Questions

Q1. Walk through what happens, node by node, when the agent above is asked "What's the weather in Tokyo?"

A: START routes to `agent`, which calls the LLM with the message history; the model responds with a tool call for `get_weather`. `should_continue` sees `tool_calls` is non-empty and routes to `tools`. `ToolNode` executes `get_weather("Tokyo")` and appends the result as a `ToolMessage`. Control returns to `agent`, which calls the LLM again — now with the tool result in context — and the model produces a final natural-language answer with no tool calls. `should_continue` now routes to `END`.

Q2. What does `ToolNode` do, and what would you have to write by hand if it didn't exist?

A: `ToolNode` inspects the last AI message for `tool_calls`, looks up each requested tool by name from the tools you gave it, executes them (in parallel if there are multiple), and returns their outputs as `ToolMessage` objects appended to state. Without it, you'd write a node that manually parses `tool_calls`, matches names to tool functions, calls each one, catches exceptions, and constructs the `ToolMessage` objects yourself.

Q3. Why does the graph route from `tools` back to `agent` unconditionally, rather than through another conditional check?

A: After a tool runs, the model always needs a chance to see the result and decide what to do next (answer, or call another tool) — that decision is the model's job, made inside the `agent` node on the next invocation. The `tools → agent` edge doesn't need to be conditional because there's only one possible next step: hand control back to the model. The actual branching decision happens afterward, in `should_continue`.

Q4. What would happen if you forgot to call `.bind_tools(tools)` on the LLM?

A: The model would never be given the tool schemas, so it could never emit a `tool_calls` field on its response — `should_continue` would always see an empty list and immediately route to `END` after the first LLM call, regardless of the tool node being wired into the graph.